Laboratorio «romperlo a propósito»: 5 fallos de contenedores y cómo diagnosticarlos

Cinco fallos típicos de los contenedores. En cada uno **provocas el fallo, lo diagnosticas con los comandos de Docker y solo después miras el arreglo**. ≈34 min en total.

> **Estado de esta guía.** Los comandos y las salidas son reales: se ejecutaron con Docker 29.4.2 sobre Ubuntu 26.04.1 LTS (GitHub Actions, 10/11/2026). **Todavía no se han ejecutado en un Codespace del curso.** En tu Codespace la versión de Docker puede ser otra: las columnas de `docker image ls`, los tiempos y los tamaños pueden diferir. Los identificadores largos (`…`) y las marcas de tiempo (`"ts": …`) se han acortado porque cambian en cada ejecución. Si algo no sale como aquí, copia el mensaje exacto: es lo que hay que comparar.

Cómo trabajar

1. Haz antes la guía práctica (ejemplos 1 a 6 como mínimo).
2. En cada rotura: ejecuta los comandos de **Provócalo y diagnostica**, anota en la hoja de registro lo que ves y **escribe tu hipótesis** antes de abrir el arreglo.
3. Despliega **Arreglo** y compara con tu hipótesis. Si no coincide, repite el diagnóstico con calma.

Las cinco roturas

Nº	Rotura	Min	Síntoma
1	El puerto ya está ocupado	5	Arrancas un segundo contenedor y Docker devuelve un error.
2	El contenedor se para nada más arrancar	8	`docker run` termina al instante y no hace lo que esperas.
3	La construcción falla: falta un archivo	8	`docker build` se detiene con un error de `COPY`.
4	El contenedor muere por falta de memoria	8	El proceso desaparece sin mensaje de error y el código de salida es 137.
5	«Arranca» pero no está en marcha	5	`docker run -d` devuelve un identificador, pero `docker ps` no muestra nada.

Rotura 1 · El puerto ya está ocupado

5 min · Síntoma: Arrancas un segundo contenedor y Docker devuelve un error.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

Provócalo y diagnostica

1. El primero arranca bien.

```
docker run -d --name uno -p 8082:8000 python:3.11-slim python -m http.server 8000
```

Salida:

```
0c9fc1f70a8b…
```

2. El segundo falla.

```
docker run -d --name dos -p 8082:8000 python:3.11-slim python -m http.server 8000
```

Este comando termina con código 125, y es lo que se espera.

Salida:

```
4569e403336f…
docker: Error response from daemon: failed to set up container networking: driver failed
programming external connectivity on endpoint dos (669256e029e1…): Bind for 0.0.0.0:8082 failed:
port is already allocated

Run 'docker run --help' for more information
```

3. Diagnóstico: `dos` se creó pero no arrancó (`Created`). El puerto 8082 de tu máquina ya lo tiene `uno`.

```
docker ps -a --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
```

Salida:

```
NAMES     STATUS                  PORTS
dos       Created
uno       Up Less than a second   0.0.0.0:8082->8000/tcp, [::]:8082->8000/tcp
```

Antes de seguir: ¿cuál es tu hipótesis de la causa? Apúntala en la hoja de registro.

▶ **Arreglo y lección** (ábrelo después de intentarlo)

Rotura 2 · El contenedor se para nada más arrancar

8 min · Síntoma: `docker run` termina al instante y no hace lo que esperas.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/R2-comando"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY app.py .
CMD ["python", "aap.py"]
```

`app.py`

```
print("la aplicacion arranco bien")
```

Provócalo y diagnostica

1. La imagen se construye sin problemas.

```
docker build -q -t r2:1 .
```

Salida:

```
sha256:d9affa0ac1ff…
```

2. Pero el contenedor no hace lo que debería.

```
docker run --name r2 r2:1
```

Este comando falla a propósito: termina con un código distinto de 0.

Salida:

```
python: can't open file '/app/aap.py': [Errno 2] No such file or directory
```

3. Diagnóstico 1: `Exited (2)`: el proceso terminó con error.

```
docker ps -a --format "table {{.Names}}\t{{.Status}}"
```

Salida:

```
NAMES     STATUS
r2        Exited (2) Less than a second ago
```

4. Diagnóstico 2: el mensaje del proceso. Busca el archivo `aap.py`: el `CMD` del Dockerfile tiene una errata.

```
docker logs r2
```

Salida:

```
python: can't open file '/app/aap.py': [Errno 2] No such file or directory
```

Antes de seguir: ¿cuál es tu hipótesis de la causa? Apúntala en la hoja de registro.

▶ **Arreglo y lección** (ábrelo después de intentarlo)

Rotura 3 · La construcción falla: falta un archivo

8 min · Síntoma: `docker build` se detiene con un error de `COPY`.

Colócate en la carpeta:

```
cd "$(git rev-parse --show-toplevel)/01-Sesiones/S2-API-y-Docker/contenedores/ejemplos/R3-copia"
```

Archivos de la carpeta (léelos antes de ejecutar nada):

`Dockerfile`

```
FROM python:3.11-slim
WORKDIR /app
COPY models/modelo.joblib models/modelo.joblib
CMD ["python", "-c", "print('el modelo esta en la imagen')"]
```

Provócalo y diagnostica

1. El `COPY` no encuentra el archivo del modelo.

```
docker build -t r3:1 .
```

Este comando falla a propósito: termina con un código distinto de 0.

Salida:

```
--------------------
   1 |     FROM python:3.11-slim
   2 |     WORKDIR /app
   3 | >>> COPY models/modelo.joblib models/modelo.joblib
   4 |     CMD ["python", "-c", "print('el modelo esta en la imagen')"]
   5 |
--------------------
ERROR: failed to build: failed to solve: failed to compute cache key: failed to calculate checksum
of ref …: "/models/modelo.joblib": not found
```

(Solo se muestran las últimas 8 líneas.)

2. Diagnóstico: el Dockerfile copia `models/modelo.joblib` y esa carpeta no existe. Es el mismo error que en el proyecto del curso si no entrenas antes de construir.

```
ls
```

Salida:

```
Dockerfile
```

Antes de seguir: ¿cuál es tu hipótesis de la causa? Apúntala en la hoja de registro.

▶ **Arreglo y lección** (ábrelo después de intentarlo)

Rotura 4 · El contenedor muere por falta de memoria

8 min · Síntoma: El proceso desaparece sin mensaje de error y el código de salida es 137.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

Provócalo y diagnostica

1. Pedimos 200 MB dentro de un contenedor limitado a 64 MB. No hay traza de Python ni mensaje: lo mata el sistema.

```
docker run --name oom --memory 64m --memory-swap 64m python:3.11-slim python -c "x = b'x' * (200 *
1024 * 1024); print('ok')"
```

Este comando termina con código 137, y es lo que se espera.

2. Diagnóstico 1: `Exited (137)` = 128 + 9: terminado con la señal 9 (KILL).

```
docker ps -a --format "table {{.Names}}\t{{.Status}}"
```

Salida:

```
NAMES     STATUS
oom       Exited (137) Less than a second ago
```

3. Diagnóstico 2: el código es 137. El indicador `oom` (*Out Of Memory*) puede salir `true` o `false` según la versión de Docker y del kernel: **no te fíes solo de él**. La prueba definitiva es repetir con más memoria (abajo).

```
docker inspect --format "estado={{.State.Status}} salida={{.State.ExitCode}} oom=
{{.State.OOMKilled}}" oom
```

Salida:

```
estado=exited salida=137 oom=true
```

Antes de seguir: ¿cuál es tu hipótesis de la causa? Apúntala en la hoja de registro.

▶ **Arreglo y lección** (ábrelo después de intentarlo)

Rotura 5 · «Arranca» pero no está en marcha

5 min · Síntoma: `docker run -d` devuelve un identificador, pero `docker ps` no muestra nada.

Colócate en la carpeta:

```
mkdir -p ~/practica-docker && cd ~/practica-docker
```

Provócalo y diagnostica

1. Arranca sin error.

```
docker run -d --name vacio python:3.11-slim
```

Salida:

```
95d2773e05e4…
```

2. No aparece: no está en marcha.

```
docker ps
```

Salida:

```
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

3. Diagnóstico: `Exited (0)` terminó **bien**… y enseguida.

```
docker ps -a --format "table {{.Names}}\t{{.Status}}"
```

Salida:

```
NAMES     STATUS
vacio     Exited (0) 1 second ago
```

4. Sin salida: no hizo nada. Ejecutó `python` sin ningún script y sin terminal, así que el intérprete terminó al instante.

```
docker logs vacio
```

Antes de seguir: ¿cuál es tu hipótesis de la causa? Apúntala en la hoja de registro.

▶ **Arreglo y lección** (ábrelo después de intentarlo)

Hoja de registro

Cópiala y rellénala mientras trabajas.

Rotura	Primer comando de diagnóstico	Qué viste	Mi hipótesis	Causa real
1. El puerto ya está ocupado				
2. El contenedor se para nada más arrancar				
3. La construcción falla: falta un archivo				
4. El contenedor muere por falta de memoria				
5. «Arranca» pero no está en marcha				

Método de diagnóstico (el mismo en las cinco)

1. **¿Está en marcha?** `docker ps -a` → ¿`Up` o `Exited (código)`?
2. **¿Qué dijo?** `docker logs nombre` → el mensaje del proceso.

3. **¿Cómo está configurado?** `docker inspect nombre` → estado, variables, límites.
4. **¿Qué hay dentro?** `docker exec nombre comando` (solo si sigue en marcha).
5. **Cambia una cosa, repite y comprueba.**